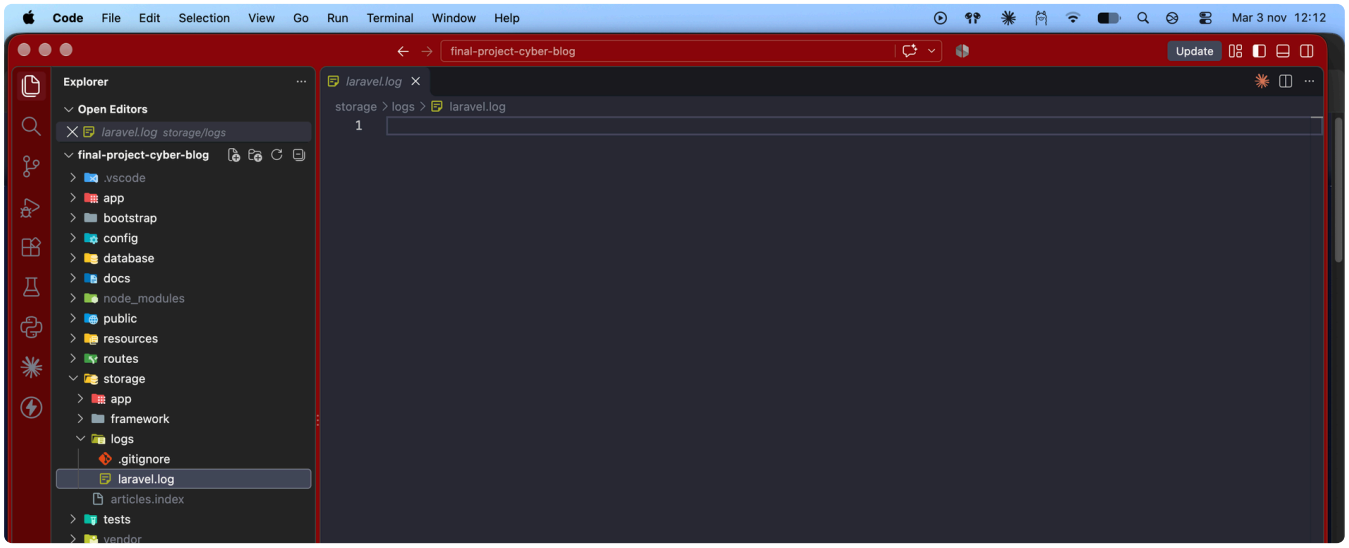
Challenge 3 — Log Mancanti per Operazioni Critiche

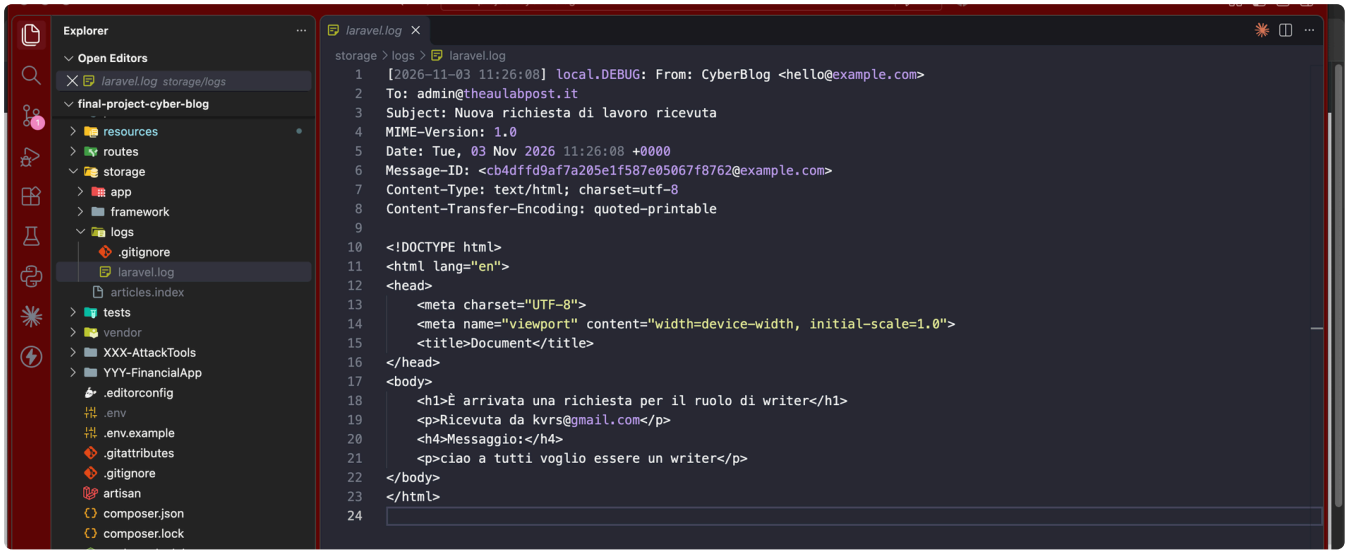
Nov 4, 2026 · @Danilo Noccioli

Il progetto di partenza non lasciava alcuna traccia delle operazioni critiche: senza log non si può ricostruire chi ha fatto cosa, quando e da dove. Ora autenticazione, articoli, cambi di ruolo ed eventi sospetti scrivono una riga con utente, azione e IP, a livelli diversi in base alla gravità. I log stanno fuori dal progetto, ruotano ogni giorno e non contengono più l'e-mail simulata della pagina careers.

Attacco (dimostrazione dell'assenza)

Non è un attacco nuovo ma la conseguenza degli altri: senza log, un DoS o un cambio di ruolo non sono attribuibili a nessuno. Nel progetto di partenza il file di log è vuoto (figura 1). Dopo aver creato un articolo e averlo approvato, l'unica voce presente è l'e-mail simulata della pagina careers (figura 2): nessun utente, nessuna azione, nessun IP.





Mitigazione

Ogni operazione critica ha un punto di log con gli stessi campi: chi (`user_id`), cosa (il messaggio), da dove (`ip`), più il dettaglio dell'evento. L'orario lo scrive Laravel in ogni riga. Mai password, mai il contenuto degli articoli.

1 — Autenticazione: un subscriber sugli eventi di Laravel

Una sola classe in `app/Listeners` ascolta gli eventi di login, logout, registrazione e login fallito, senza toccare Fortify. Laravel la collega da solo grazie alla scoperta automatica dei metodi `handle...`, verificata con `php artisan event:list`.

```
class AuthEventSubscriber
{
    public function handleUserLogin(Login $event): void
    {
        $user = $event->user;

        Log::info('User logged in', [
            'user_id' => $user->id,
            'ip' => request()->ip(),
        ]);
    }

    public function handleUserLogout(Logout $event): void
    {
        Log::info('User logged out', [
            'user_id' => $event->user?->id,
            'ip' => request()->ip(),
        ]);
    }

    public function handleUserLoginFailed(Failed $event): void
    {
        Log::warning('User login failed', [
            'email' => $event->credentials['email'] ?? null,
            'ip' => request()->ip(),
        ]);
    }

    public function handleUserRegistered(Registered $event): void
    {
        Log::info('User registered', [
            'user_id' => $event->user->id,
```

```
            'ip' => request()->ip(),
        ]);
    }
}
```

2 — Blocco del brute force: il log sta nel rate limiter

La rotta di login passa da `ThrottleRequests:login`, che risponde 429 prima che Fortify lanci l'evento `Lockout`: per questo il blocco non è visibile al subscriber. Il log è quindi nella risposta del limiter, che deve comunque restituire una risposta HTTP valida. Il metodo `handleUserLockout` resta nel subscriber per il caso in cui il blocco passasse dal controllo interno di Fortify. È il legame con la Challenge 1: il limiter ferma l'attacco, il log dice chi lo ha tentato.

```
RateLimiter::for('login', function (Request $request) {
    $throttleKey = Str::transliterate(Str::lower($request-
>input(Fortify::username())).'|'.$request->ip());

    return Limit::perMinute(5)->by($throttleKey)->response(function (Request
$request, array $headers) {
        Log::error('User lockout: too many failed login attempts', [
            'email' => request()->input(Fortify::username()),
            'ip' => request()->ip(),
        ]);
        return response('Too many Requests', 429, $headers);
    });
});
```

3 — Articoli: creazione, modifica, eliminazione e tentativi altrui

Una riga `info` dopo ogni operazione riuscita in `store()`, `update()` e `destroy()` (qui `store()`), con l'id dell'articolo al posto del contenuto. In `edit()` l'accesso negato a un articolo non proprio è un evento sospetto e usa `warning`.

```
// ArticleController@store
Log::info('Articolo creato', [
    'user_id' => Auth::id(),
    'ip' => $request->ip(),
    'article_id' => $article->id,
]);

// ArticleController@edit
if (Auth::user()->id != $article->user_id) {
    Log::warning('Tentativo di modifica di un articolo altrui', [
        'user_id' => Auth::id(),
        'ip' => request()->ip(),
        'article_id' => $article->id,
        'owner_id' => $article->user_id,
    ]);

    return redirect()->route('homepage')->with('alert', 'Accesso non
consentito');
}
```

4 — Cambi di ruolo con valore prima/dopo

Il valore precedente va letto prima della modifica, altrimenti il log mostrerebbe solo quello nuovo. `before` a `null` indica una richiesta di ruolo ancora in attesa. La promozione ad admin usa `warning` perché dà accesso a tutto; writer e revisor usano `info`.

```
// AdminController@setAdmin
public function setAdmin(User $user)
{
    $previousValue = $user->is_admin;

    $user->is_admin = true;
    $user->save();

    Log::warning('User promoted to admin', [
        'actor_id' => Auth::id(),
        'target_id' => $user->id,
        'ip' => request()->ip(),
        'before' => $previousValue,
        'after' => $user->is_admin,
    ]);

    return redirect(route('admin.dashboard'))->with('message', "$user->name
is now administrator");
}
```

5 — Accessi non autorizzati alle rotte admin

I due middleware respingevano con un redirect senza lasciare traccia. Ora il log è scritto nel ramo che blocca, con messaggi diversi per distinguere il controllo sul ruolo da quello sulla rete (header `Host`).

```
// UserIsAdmin
public function handle(Request $request, Closure $next): Response
{
    if(Auth::user() && Auth::user()->is_admin){
        return $next($request);
    }
    Log::warning('Unauthorized access attempt to admin route', [
        'user_id' => Auth::user()?->id,
        'ip' => $request->ip(),
        'route' => $request->path(),
    ]);
    return redirect(route('homepage'))->with('alert', 'Not Authorized');
}

// OnlyLocalAdmin
public function handle(Request $request, Closure $next): Response
{
    $allowedHosts = ['internal.admin:8000'];

    $host = $request->header('Host');

    if (! in_array($host, $allowedHosts)) {
        Log::warning('Unauthorized access attempt to local admin route', [
            'user_id' => auth()->user()?->id,
            'host' => $host,
            'ip' => $request->ip(),
            'route' => $request->path(),
        ]);

        return redirect(route('homepage'))->with('alert', 'Not Authorized');
    }

    return $next($request);
}
```

Livelli e campi

Evento	Livello	Campi
Login, logout, registrazione	`info`	`user_id`, `ip`
Articolo creato, modificato, cancellato	`info`	`user_id`, `ip`, `article_id`
Promozione a writer o revisor	`info`	`actor_id`, `target_id`, `ip`, `before`, `after`
Promozione a admin	`warning`	come sopra
Login fallito	`warning`	`email`, `ip`
Modifica di un articolo altrui	`warning`	`user_id`, `ip`, `article_id`, `owner_id`
Accesso admin non autorizzato (ruolo o host)	`warning`	`user_id`, `ip`, `route` (+ `host`)
Blocco per troppi tentativi di login	`error`	`email`, `ip`

6 — Dove vivono i log (regole d'oro)

- **Fuori dal progetto:** il file non sta più in `storage/logs` ma in una cartella esterna, letta dal `.env` tramite `LOG_PATH`. Un deploy, un backup o un errore con git non possono sovrascriverlo, e chi compromette l'applicazione non lo trova insieme al codice.
- **Rotazione:** canale `daily`, un file per giorno e conservazione di 14 giorni (`LOG_DAILY_DAYS`).
- **Niente dati sensibili:** niente password, niente contenuto degli articoli, nessun dato finanziario. Nei login falliti si registra l'e-mail tentata, che serve a capire chi è preso di mira.
- **E-mail fuori dal log:** con `MAIL_MAILER=log` ogni e-mail veniva scritta nel file di log, con indirizzo e testo del candidato (figura 2). Il mailer di sviluppo è ora `array`: l'app funziona ma nulla di quel messaggio finisce nel log. Nessuna e-mail viene davvero consegnata: in produzione serve un SMTP reale.

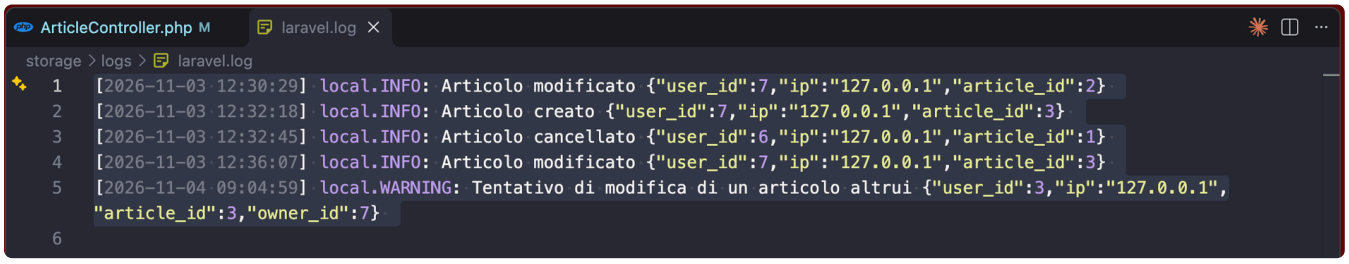
```
// config/logging.php
'daily' => [
    'driver' => 'daily',
    'path' => env('LOG_PATH', storage_path('logs/laravel.log')),
    'level' => env('LOG_LEVEL', 'debug'),
    'days' => env('LOG_DAILY_DAYS', 14),
    'replace_placeholders' => true,
],
```

```
# .env
LOG_CHANNEL=stack
LOG_STACK=daily
LOG_LEVEL=debug
LOG_PATH=/percorso/fuori/dal/progetto/cyberblog-logs/laravel.log
MAIL_MAILER=array
```

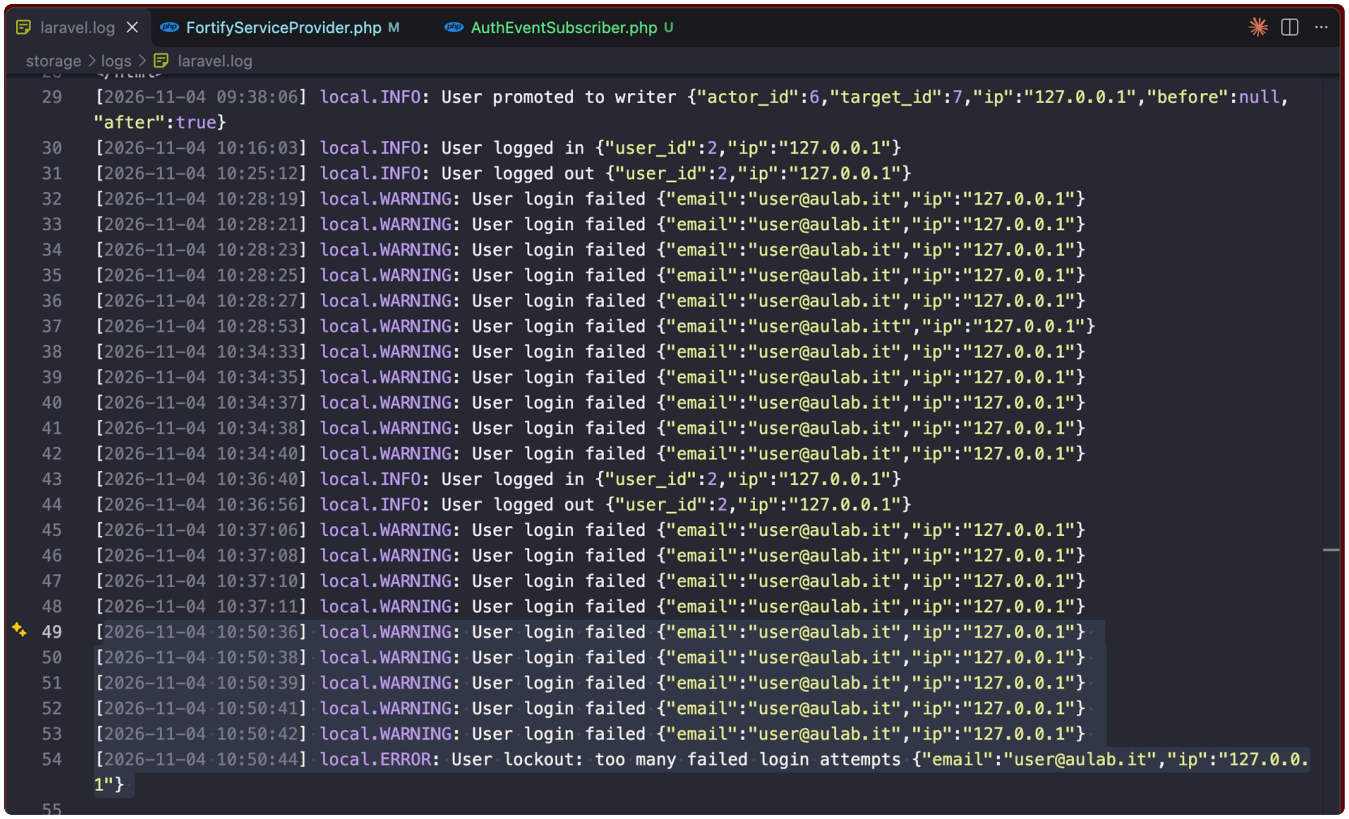
Verifica

Ripetute le azioni critiche e i tentativi di attacco, il log contiene ora la traccia con utente, azione, orario e IP. Gli orari sono in UTC (in Italia +1 ora).

Articoli. Creazione (utente 7, articolo 3), modifica, eliminazione (utente 6, articolo 1) e il tentativo dell'utente 3 di modificare l'articolo 3, che appartiene all'utente 7: un `WARNING` con `owner_id` (figura 3).



Ruoli e autenticazione. Promozione a writer da parte dell'admin 6 verso l'utente 7 (`before: null`, `after: true`), login e logout, poi una serie di login falliti sulla stessa e-mail. Al sesto tentativo, alle 10:50:44, scatta il blocco in `ERROR` (figura 4).



Verifica finale sul file esterno al progetto. Promozione a revisor, brute force con blocco e accesso dell'utente 4, non admin, alla rotta `admin/dashboard`: respinto e tracciato (figura 5).



Il middleware sull'host ha prodotto la riga corrispondente quando l'admin 5 ha aperto la dashboard da `127.0.0.1:8000` invece di `internal.admin:8000`:

```
[2026-11-04 11:11:53] local.WARNING: Unauthorized access attempt to local admin route
{"user_id":5,"host":"127.0.0.1:8000","ip":"127.0.0.1","route":"admin/dashboard"}
```